About this Book

This book introduces matrices through high school coordinate geometry. This approach makes it easier for beginners to learn Python for scientific computing. All problems in the book are from NCERT mathematics textbooks from Class 9-12. Exercises are from CBSE board exam papers.

The content is sufficient for industry jobs and covers nearly all matrix prerequisites for machine learning. There is no copyright, so readers are free to print and share.

License: https://creativecommons.org/licenses/by-sa/3.0/
and
https://www.gnu.org/licenses/fdl-1.3.en.html

First manual appeared in January 2018
First edition published on July 10, 2024

In this edition, some incorrect solutions were removed. All figures redrawn. More problems added.

CONTENTS

1 GATE CH 2025 3
1.1 Formulae 3
1.2 Problems 9

1. GATE CH 2025

A. Formulae

1.1.1 Dot Product of Two Vectors

- Matrix and Summation Forms: For two n-dimensional vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\mathbf{a}^\top \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{1.1.1.1}$$

1.1.2 Complex Numbers and Arguments

- Vector Representation of a Complex Number:
  A complex number

$$z = a + bi \tag{1.1.2.1}$$

  can be represented by

$$\mathbf{z} = \begin{pmatrix} a \\ b \end{pmatrix}. \tag{1.1.2.2}$$

- Matrix Representation of Complex Multiplication:
  If

$$z_1 = a + bi, \qquad z_2 = c + di, \tag{1.1.2.3}$$

  then

$$\begin{pmatrix} \mathrm{Re}\,(z_1 z_2) \\ \mathrm{Im}\,(z_1 z_2) \end{pmatrix} = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} c \\ d \end{pmatrix}. \tag{1.1.2.4}$$

- Argument from the Dot Product:
  Let

$$\mathbf{z} = \begin{pmatrix} x \\ y \end{pmatrix} \tag{1.1.2.5}$$

  represent a nonzero complex number and let

$$\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{1.1.2.6}$$

  be the unit vector along the positive real axis.
  Then

$$\cos\theta = \frac{\mathbf{z}^\top \hat{i}}{\|\mathbf{z}\|\,\|\hat{i}\|}. \tag{1.1.2.7}$$

  The signs of x and y determine the quadrant of the resultant vector.

1.1.3 Mean Residence Time from RTD

- Mean Residence Time: For a residence time distribution $E(t)$,

$$\bar{t} = \frac{\int_0^\infty tE(t)\ dt}{\int_0^\infty E(t)\ dt} \tag{1.1.3.1}$$

$$\int_0^\infty E(t)\ dt = 1 \tag{1.1.3.2}$$

$$\bar{t} = \int_0^\infty tE(t)\ dt \tag{1.1.3.3}$$

1.1.4 Permutations and Combinations

- Number of Ways of Choosing r Items from n Items:

$${}^nC_r = \binom{n}{r} = \frac{n!}{r!\,(n-r)!} \tag{1.1.4.1}$$

1.1.5 Expectation of a Function

- Continuous Random Variable:
  For a continuous random variable X with probability density function $f_X(x)$,

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x)\, f_X(x)\ dx \tag{1.1.5.1}$$

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x)\ dx \tag{1.1.5.2}$$

- Discrete Random Variable:

$$\mathbb{E}[g(X)] = \sum_x g(x) \Pr(X = x) \tag{1.1.5.3}$$

1.1.6 Newton-Raphson Method

- First-Order Recurrence:
  For a differentiable function $f(x)$, the root is iteratively approximated using the Newton-Raphson recurrence

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{1.1.6.1}$$

1.1.7 Eigenvalues, Eigenvectors and Trace

- Definition:
  For a square matrix $\mathbf{A}$,

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}, \qquad \mathbf{x} \neq \mathbf{0} \tag{1.1.7.1}$$

  where λ is an eigenvalue and $\mathbf{x}$ is the corresponding eigenvector.
- Characteristic Equation:

$$\left|\mathbf{A} - \lambda\mathbf{I}\right| = 0 \tag{1.1.7.2}$$

- Relation with Trace:

If $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\operatorname{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i = \lambda_1 + \lambda_2 + \cdots + \lambda_n \tag{1.1.7.3}$$

Also, from the eigenvalue product relation,

$$\left|\mathbf{A}\right| = \prod_{i=1}^{n} \lambda_i \tag{1.1.7.4}$$

1.1.8 Trapezoidal Method for Plotting Differential Equations

- Differential Equation:

$$\frac{dy}{dx} = f(x, y) \tag{1.1.8.1}$$

- Trapezoidal Update:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(x_n, y_n) + f(x_{n+1}, y_{n+1})\right] \tag{1.1.8.2}$$

- The method approximates the solution by using the average of the slopes at two consecutive points. The calculated points can then be used to plot the numerical solution curve.

1.1.9 Jacobian Matrix and Divergence

- Jacobian Matrix:
  For the vector field $\mathbf{F} = \begin{pmatrix} F_1 & F_2 & \cdots & F_n \end{pmatrix}^\top$ with variables $\mathbf{x} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}^\top$,

$$J_{\mathbf{F}} = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} & \cdots & \frac{\partial F_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \frac{\partial F_n}{\partial x_2} & \cdots & \frac{\partial F_n}{\partial x_n} \end{pmatrix} \tag{1.1.9.1}$$

- Divergence:

$$\nabla \cdot \mathbf{F} = \sum_{i=1}^{n} \frac{\partial F_i}{\partial x_i} \tag{1.1.9.2}$$

- Relation Between Jacobian and Divergence:

$$\nabla \cdot \mathbf{F} = \operatorname{tr}(J_{\mathbf{F}}) \tag{1.1.9.3}$$

1.1.10 Dead Time and Time Delay in the Laplace Domain

- Time Delay:
  A pure time delay of θ in the time domain is represented in the Laplace domain by

$$e^{-\theta s} \tag{1.1.10.1}$$

- Delayed Signal:
  If a signal $x(t)$ is delayed by θ, then

$$x_d(t) = x(t-\theta)\,u(t-\theta) \tag{1.1.10.2}$$

$$X_d(s) = e^{-\theta s}X(s) \tag{1.1.10.3}$$

- Significance in a Transfer Function:
  For a process transfer function

$$G(s) = G_0(s)\,e^{-\theta s}, \tag{1.1.10.4}$$

  the factor $e^{-\theta s}$ represents a pure time delay or dead time of θ time units, while $G_0(s)$ represents the process dynamics without the delay.
- The dead time shifts the process response along the time axis by θ without changing the steady-state gain. For a first-order plus dead-time model,

$$G_{\text{FOPDT}}(s) = \frac{Ke^{-\theta s}}{\tau s + 1} \tag{1.1.10.5}$$

  where K is the process gain, τ is the time constant, and θ is the dead time. This is the same FOPDT form given in (1.1.11.1).

1.1.11 Maximum Slope Method for FOPDT Approximation

- First-Order Plus Dead-Time Model:

$$G(s) = \frac{Ke^{-\theta s}}{\tau s + 1} \tag{1.1.11.1}$$

- Process Gain (from the FOPDT model in (1.1.11.1)):

$$K = \frac{\Delta y}{\Delta u} \tag{1.1.11.2}$$

- Maximum Slope:

$$R = \max\left(\frac{dy}{dt}\right) \tag{1.1.11.3}$$

- Dead Time:
  Using the maximum-slope construction in (1.1.11.3), if $(t_{\text{inflection}}, y_{\text{inflection}})$ is the inflection point,

$$t_i = t_{\text{inflection}} - \frac{y_{\text{inflection}} - y_0}{R} \tag{1.1.11.4}$$

$$\theta = t_i - t_0 \tag{1.1.11.5}$$

- Time Constant:
  Using the process gain and maximum slope definitions in (1.1.11.2) and (1.1.11.3),

$$\tau = \frac{K\Delta u}{R} \tag{1.1.11.6}$$

1.1.12 Transfer Function and Feedback Loop

- Transfer Function:

$$G(s) = \frac{Y(s)}{X(s)} \tag{1.1.12.1}$$

$$Y(s) = G(s) X(s) \tag{1.1.12.2}$$

- Negative Feedback:
For forward transfer function $G(s)$ and feedback transfer function $H(s)$,

$$E(s) = X(s) - H(s) Y(s) \tag{1.1.12.3}$$

$$Y(s) = G(s) E(s) \tag{1.1.12.4}$$

- Closed-Loop Transfer Function:
Using the relations in (1.1.12.3) and (1.1.12.4),

$$Y(s) = G(s) [X(s) - H(s) Y(s)] \tag{1.1.12.5}$$

$$Y(s) [1 + G(s) H(s)] = G(s) X(s) \tag{1.1.12.6}$$

$$\frac{Y(s)}{X(s)} = \frac{G(s)}{1 + G(s) H(s)} \tag{1.1.12.7}$$

- Unity Feedback:
Setting the feedback transfer function to unity in (1.1.12.7),

$$H(s) = 1 \tag{1.1.12.8}$$

$$\frac{Y(s)}{X(s)} = \frac{G(s)}{1 + G(s)} \tag{1.1.12.9}$$

1.1.13 Routh-Hurwitz Stability Criterion

- Characteristic Equation:
For a characteristic polynomial

$$a_n s^n + a_{n-1} s^{n-1} + \cdots + a_1 s + a_0 = 0 \tag{1.1.13.1}$$

- Routh Array:

$$\begin{pmatrix} a_n & a_{n-2} & a_{n-4} & \cdots \\ a_{n-1} & a_{n-3} & a_{n-5} & \cdots \\ b_1 & b_2 & b_3 & \cdots \\ c_1 & c_2 & c_3 & \cdots \end{pmatrix} \tag{1.1.13.2}$$

- First Elements:
From the Routh array structure in (1.1.13.2), the first elements are

$$b_1 = \frac{a_{n-1} a_{n-2} - a_n a_{n-3}}{a_{n-1}} \tag{1.1.13.3}$$

$$b_2 = \frac{a_{n-1} a_{n-4} - a_n a_{n-5}}{a_{n-1}} \tag{1.1.13.4}$$

- Number of Unstable Poles:

$$N_{\text{RHP}} = \text{number of sign changes in the first column} \tag{1.1.13.5}$$

- Stability Conditions:

$$\text{Stable system} \iff \text{no sign changes in the first column} \tag{1.1.13.6}$$

$$\text{Unstable system} \iff \text{one or more sign changes in the first column} \tag{1.1.13.7}$$

$$\text{Number of unstable poles} = N_{\text{RHP}} \tag{1.1.13.8}$$

B. Problems

1.2.1 Consider a Cartesian coordinate system defined over a 3-dimensional vector space with orthogonal unit basis vectors $\hat{i}$, $\hat{j}$ and $\hat{k}$. Let

$$\mathbf{a} = \sqrt{2}\hat{i} + \frac{1}{\sqrt{2}}\hat{j} + \hat{k}, \tag{1.2.1.1}$$

$$\mathbf{b} = \frac{1}{\sqrt{2}}\hat{i} + \sqrt{2}\hat{j} - \hat{k}. \tag{1.2.1.2}$$

The inner product of these vectors $(\mathbf{a} \cdot \mathbf{b})$ is (GATE CH 2025)

a) 0 b) 1 c) -1 d) 2

Solution: The dot product of two vectors is given by (1.1.1.1):

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} \tag{1.2.1.3}$$

$$= \begin{pmatrix} a_x & a_y & a_z \end{pmatrix} \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} \tag{1.2.1.4}$$

$$= a_x b_x + a_y b_y + a_z b_z. \tag{1.2.1.5}$$

Substituting the components of the given vectors,

$$\mathbf{a} \cdot \mathbf{b} = \left(\sqrt{2}\right)\left(\frac{1}{\sqrt{2}}\right) + \left(\frac{1}{\sqrt{2}}\right)\left(\sqrt{2}\right) + (1)(-1) \tag{1.2.1.6}$$

$$= 1 + 1 - 1 \tag{1.2.1.7}$$

$$= 1. \tag{1.2.1.8}$$

The code to verify this is:

codes/q12/main.py

1.2.2 Consider two complex numbers $z_1 = 1 - i$ and $z_2 = i$. The argument of $z_1 z_2$ is (GATE CH 2025)

a) 0 b) $\frac{\pi}{4}$ c) $\frac{\pi}{2}$ d) π

Solution:
A complex number

$$z = a + bi \tag{1.2.2.1}$$

can be represented by the vector

$$\mathbf{z} = \begin{pmatrix} a \\ b \end{pmatrix}. \tag{1.2.2.2}$$

If

$$z_1 = a + bi, \qquad z_2 = c + di, \tag{1.2.2.3}$$

then multiplication by z_1 can be represented by

$$\begin{pmatrix} \mathrm{Re}\,(z_1 z_2) \\ \mathrm{Im}\,(z_1 z_2) \end{pmatrix} = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} c \\ d \end{pmatrix} \tag{1.2.2.4}$$

For the given numbers,

$$z_1 = 1 - i, \qquad z_2 = i. \tag{1.2.2.5}$$

Therefore,

$$\begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \tag{1.2.2.6}$$

Using the transformation matrix, (1.1.2.4),

$$\mathbf{z_1 z_2} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{1.2.2.7}$$

$$= \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \tag{1.2.2.8}$$

Thus, the resultant vector is

$$\mathbf{z_1 z_2} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \tag{1.2.2.9}$$

Since both components are positive, the resultant vector lies in the first quadrant. The unit vector along the positive real axis is

$$\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \|\hat{i}\| = 1. \tag{1.2.2.10}$$

Using the dot-product relation,

$$\cos\theta = \frac{\mathbf{z_1 z_2}^{\top} \hat{i}}{\|\mathbf{z_1 z_2}\|\,\|\hat{i}\|}. \tag{1.2.2.11}$$

Now,

$$\mathbf{z_1 z_2}^{\top} \hat{i} = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{1.2.2.12}$$

$$= 1, \tag{1.2.2.13}$$

and

$$\|\mathbf{z_1 z_2}\| = \sqrt{1^2 + 1^2} \tag{1.2.2.14}$$

$$= \sqrt{2}. \tag{1.2.2.15}$$

Therefore,

$$\cos\theta = \frac{1}{\sqrt{2}}. \tag{1.2.2.16}$$

Since the resultant vector lies in the first quadrant,

$$0 < \theta < \frac{\pi}{2}. \tag{1.2.2.17}$$

Hence,

$$\theta = \cos^{-1}\left(\frac{1}{\sqrt{2}}\right) \tag{1.2.2.18}$$

$$= \frac{\pi}{4}. \tag{1.2.2.19}$$

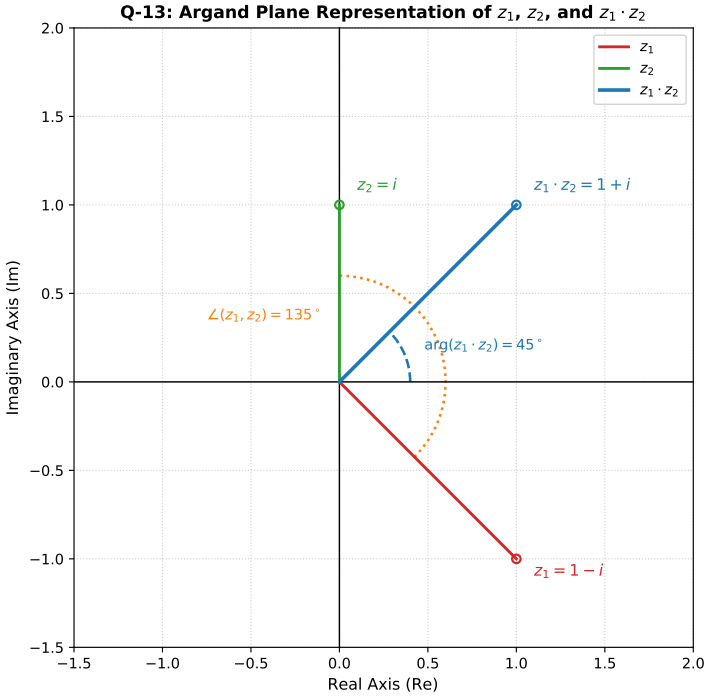


Fig. 1.2.2.1. Complex-plane representation of $z_1 z_2$

The code to verify this is:

codes/q13/main.py

1.2.3 A box contains 3 identical green balls and 7 identical blue balls. Two balls are randomly drawn without replacement from the box. The probability of drawing 1 green and 1 blue ball is (GATE CH 2025)

a) $\frac{^3P_1 \times ^7P_1}{^{10}P_2}$ b) $\frac{^{10}P_3 \times ^{10}P_7}{^{10}P_2}$ c) $\frac{^{10}C_3 \times ^{10}C_7}{^{10}C_2}$ d) $\frac{^3C_1 \times ^7C_1}{^{10}C_2}$

Solution: There are 3+7=10 balls in total. We need one green ball and one blue ball. Using the combination formula in (1.1.4.1), the number of favourable selections is

$$^3C_1 \, ^7C_1. \tag{1.2.3.1}$$

The total number of ways of selecting two balls from ten balls is

$$^{10}C_2. \tag{1.2.3.2}$$

Therefore,

$$P\left(1 \text{ green}, 1 \text{ blue}\right) = \frac{^3C_1 \, ^7C_1}{^{10}C_2} \tag{1.2.3.3}$$

$$= \frac{3 \times 7}{\frac{10 \times 9}{2}} \tag{1.2.3.4}$$

$$= \frac{21}{45} \tag{1.2.3.5}$$

$$= \frac{7}{15}. \tag{1.2.3.6}$$

The code and data to verify this are:

codes/q14/main.c

codes/q14/main.dat

1.2.4 The residence time distribution function $E\left(t\right)$ for a reactor is given by

$$E\left(t\right) = \begin{cases} 1 - 2t, & t \leq 0.5 \text{ min}, \\ 0, & t > 0.5 \text{ min}. \end{cases} \tag{1.2.4.1}$$

The mean residence time $(inmin)$ is _____ (rounded off to two decimal places). (GATE CH 2025)

Solution: Using the mean residence-time relation in (1.1.3.1),

$$\bar{t} = \frac{\int_0^\infty tE\left(t\right) \, dt}{\int_0^\infty E\left(t\right) \, dt}. \tag{1.2.4.2}$$

Since $E(t) = 0$ for $t > 0.5$,

$$\bar{t} = \frac{\int_0^{0.5} t\,(1-2t)\;dt}{\int_0^{0.5} (1-2t)\;dt}. \tag{1.2.4.3}$$

For the numerator,

$$\int_0^{0.5} t\,(1-2t)\;dt = \int_0^{0.5} \left(t - 2t^2\right)\;dt \tag{1.2.4.4}$$

$$= \left[\frac{t^2}{2} - \frac{2t^3}{3}\right]_0^{0.5} \tag{1.2.4.5}$$

$$= \frac{1}{8} - \frac{1}{12} \tag{1.2.4.6}$$

$$= \frac{1}{24}. \tag{1.2.4.7}$$

For the denominator,

$$\int_0^{0.5} (1-2t)\;dt = \left[t - t^2\right]_0^{0.5} \tag{1.2.4.8}$$

$$= \frac{1}{2} - \frac{1}{4} \tag{1.2.4.9}$$

$$= \frac{1}{4}. \tag{1.2.4.10}$$

Hence,

$$\bar{t} = \frac{1/24}{1/4} = \frac{1}{6} = 0.1667 \text{ min} \tag{1.2.4.11}$$

$$\approx 0.17 \text{ min}. \tag{1.2.4.12}$$

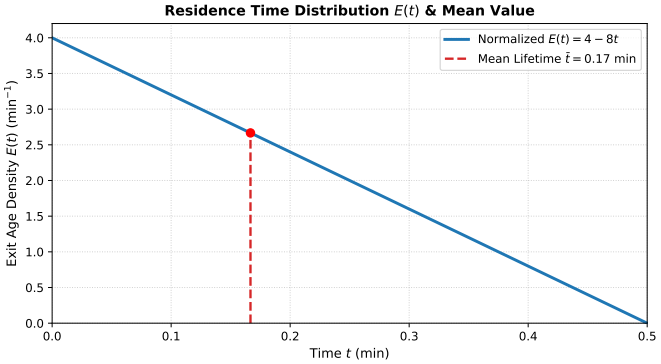


Fig. 1.2.4.1. Residence time distribution $E(t)$ for the given reactor

The code to verify this is:

codes/q35/main.py

1.2.5 Consider a Cartesian coordinate system in the $x-y$ plane, where $x, y \in [0,1]$. Let the vector be

$$\mathbf{v} = ax\hat{i} - by\hat{j}. \tag{1.2.5.1}$$

Which one of the following conditions is required for the divergence of $\mathbf{v}$ to be zero? (GATE CH 2025)

a) $a-b=0$ b) $a<b$ c) $a>b$ d) $a+b=0$

Solution: For a vector field, the Jacobian matrix is given by (1.1.9.1), and the divergence is related to its trace by (1.1.9.3).
For

$$\mathbf{v} = v_x\hat{i} + v_y\hat{j}, \tag{1.2.5.2}$$

the Jacobian is

$$\begin{pmatrix} \frac{\partial v_x}{\partial x} & \frac{\partial v_x}{\partial y} \\ \frac{\partial v_y}{\partial x} & \frac{\partial v_y}{\partial y} \end{pmatrix} \tag{1.2.5.3}$$

The divergence is the trace:

$$\nabla \cdot \mathbf{v} = \mathrm{tr}\,(J_{\mathbf{v}}) \tag{1.2.5.4}$$

$$= \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y}. \tag{1.2.5.5}$$

Here,

$$v_x = ax, \qquad v_y = -by. \tag{1.2.5.6}$$

Therefore,

$$\frac{\partial v_x}{\partial x} = a, \qquad \frac{\partial v_y}{\partial y} = -b. \tag{1.2.5.7}$$

Thus,

$$\nabla \cdot \mathbf{v} = a - b. \tag{1.2.5.8}$$

For zero divergence,

$$a - b = 0. \tag{1.2.5.9}$$

The code to verify this is:

codes/q36/main.py

1.2.6 Let X be a continuous random variable with probability density function

$$p\left(x\right)=\begin{cases}\dfrac{1}{a}, & x\in\left(0,a\right),\\ 0, & \text{otherwise},\end{cases} \tag{1.2.6.1}$$

where a is a positive constant. Let $f\left(x\right)=x^2$. The expected value of $f\left(X\right)$ is (GATE CH 2025)

a) $\dfrac{a^2}{3}$ b) $\dfrac{a^3}{3}$ c) $\dfrac{2a^2}{3}$ d) $\dfrac{2a^3}{3}$

Solution: The given function and probability density are

$$f\left(x\right)=x^2, \tag{1.2.6.2}$$

$$p\left(x\right)=\frac{1}{a}, \qquad 0<x<a. \tag{1.2.6.3}$$

The expected value of a function of a continuous random variable is obtained from (1.1.5.1), which is the continuous form of taking a weighted average over all possible outcomes:

$$E[f\left(X\right)]=\int_{-\infty}^{\infty}f\left(x\right)p\left(x\right)\,dx. \tag{1.2.6.4}$$

Since the density is non-zero only from 0 to a,

$$E[x^2]=\int_0^a x^2\frac{1}{a}\,dx \tag{1.2.6.5}$$

$$=\frac{1}{a}\left[\frac{x^3}{3}\right]_0^a \tag{1.2.6.6}$$

$$=\frac{1}{a}\frac{a^3}{3} \tag{1.2.6.7}$$

$$=\frac{a^2}{3}. \tag{1.2.6.8}$$

The expected value represents the average output obtained when the possible outcomes are weighted according to their probabilities.

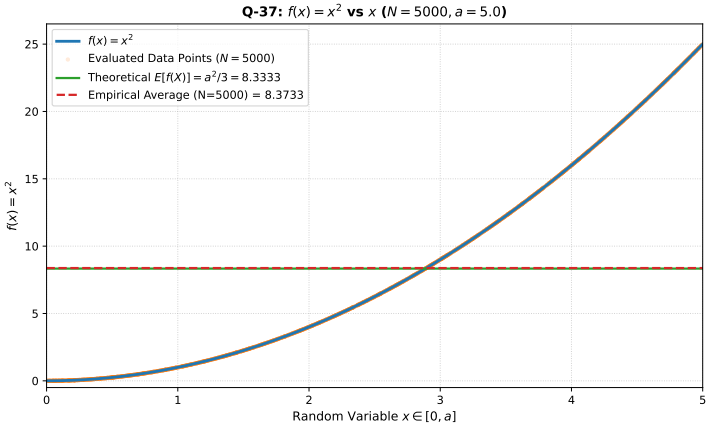


Fig. 1.2.6.1. Uniform probability density over the interval $(0, a)$

The codes and data to verify this are:

codes/q37/main.dat

codes/q37/main.py

codes/q37/main.c

1.2.7 The differential equation

$$\frac{dy}{dx} = -\frac{y}{x}$$

has which of the following correct solution(s)? (GATE CH 2025)

a) $y = x + c$ b) $y = \frac{c}{x}$ c) $y = -x + c$ d) $y = 0$

Solution:

1.2.7.1 **Theoretical Solution**

Rearranging the given differential equation,

$$\frac{dy}{dx} = -\frac{y}{x}. \tag{1.2.7.1.1}$$

Separating the variables,

$$\frac{1}{y}\,dy = -\frac{1}{x}\,dx. \tag{1.2.7.1.2}$$

Integrating both sides,

$$\int \frac{1}{y}\,dy = -\int \frac{1}{x}\,dx. \tag{1.2.7.1.3}$$

Therefore,

$$\ln|y| = -\ln|x| + C. \tag{1.2.7.1.4}$$

Hence,

$$\ln|xy| = C, \tag{1.2.7.1.5}$$

which gives

$$y = \frac{C}{x}. \tag{1.2.7.1.6}$$

For $C = 0$,

$$y = 0. \tag{1.2.7.1.7}$$

Thus, both

$$y = \frac{C}{x} \qquad \text{and} \qquad y = 0$$

are valid solutions.

1.2.7.2 **Numerical Method**

For numerical verification, the trapezoidal method can be applied to

$$\frac{dy}{dx} = f(x,y) = -\frac{y}{x}. \tag{1.2.7.2.1}$$

For a step size h, the trapezoidal approximation is

$$y_{n+1} = y_n + \frac{h}{2}\left[f(x_n,y_n) + f(x_{n+1},y_{n+1})\right]. \tag{1.2.7.2.2}$$

Substituting

$$f(x,y) = -\frac{y}{x}, \tag{1.2.7.2.3}$$

gives

$$y_{n+1} = y_n + \frac{h}{2}\left[-\frac{y_n}{x_n} - \frac{y_{n+1}}{x_{n+1}}\right]. \tag{1.2.7.2.4}$$

This relation is used iteratively to obtain the numerical solution,

which can then be compared with the theoretical solution $y = C/x$.
The trapezoidal method can then be used to generate numerical points for plotting the solution family.

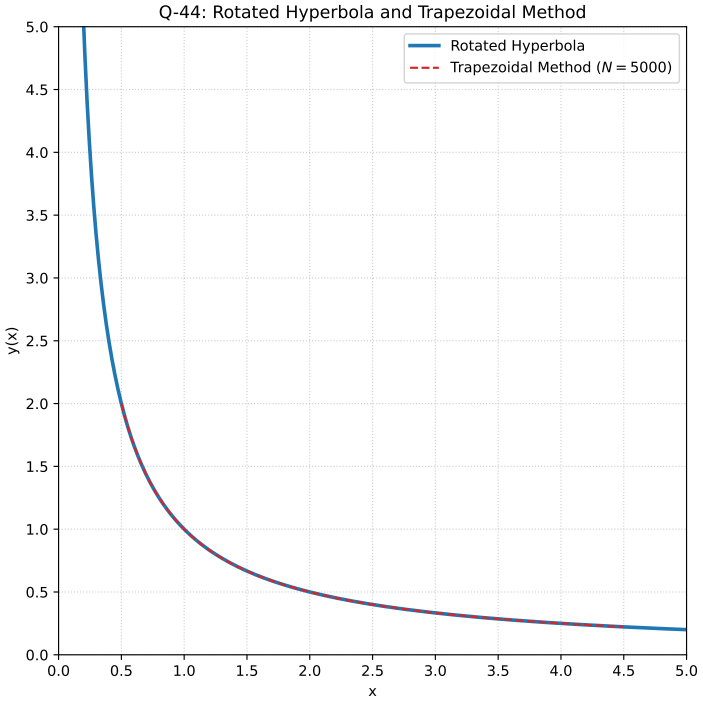


Fig. 1.2.7.1. Numerical solution of the differential equation.

The code to verify this is:

codes/q44/main.py

1.2.8 Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 1 & 2 \end{pmatrix}. \tag{1.2.8.1}$$

One of its eigenvalues is 0.27. The other eigenvalue, rounded off to two decimal places, is _____. (GATE CH 2025)

Solution: For an eigenvalue λ and its corresponding eigenvector $\mathbf{x}$,

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}, \qquad \mathbf{x} \neq \mathbf{0}. \tag{1.2.8.2}$$

The eigenvalues are obtained from the characteristic equation

$$\left|\mathbf{A} - \lambda\mathbf{I}\right| = 0. \tag{1.2.8.3}$$

From the trace relation in (1.1.7.3), the sum of the eigenvalues is

$$\lambda_1 + \lambda_2 = \left(2 - \sqrt{3}\right) + \left(2 + \sqrt{3}\right) \tag{1.2.8.4}$$

$$= 4. \tag{1.2.8.5}$$

The trace of the matrix is

$$\text{tr}\left(\mathbf{A}\right) = 2 + 2 = 4. \tag{1.2.8.6}$$

Hence,

$$\boxed{\lambda_1 + \lambda_2 = \text{tr}\left(\mathbf{A}\right)}. \tag{1.2.8.7}$$

Using the given value $\lambda_1 = 0.27$ and the trace,

$$\lambda_2 = 4 - 0.27 \tag{1.2.8.8}$$

$$= 3.73. \tag{1.2.8.9}$$

The code to verify this is:

codes/q45/main.py

1.2.9 The Newton-Raphson method is used to obtain the root of

$$f\left(x\right) = x^2 - x - 1 = 0 \tag{1.2.9.1}$$

with the initial guess $x_0 = 1$. The value of the second iterate x_2, rounded off to two decimal places, is _____. (GATE CH 2025)

Solution: Newton-Raphson improves an initial guess by using the tangent to the function at the current point. Repeating the iteration moves the approximation towards the root.

The Newton-Raphson formula is given by (1.1.6.1):

$$x_{(n+1)} = x_n - \frac{f\left(x_n\right)}{f'\left(x_n\right)}. \tag{1.2.9.2}$$

Here,

$$f\left(x\right) = x^2 - x - 1, \tag{1.2.9.3}$$

$$f'\left(x\right) = 2x - 1. \tag{1.2.9.4}$$

Given

$$x_0 = 1. \tag{1.2.9.5}$$

For the first iteration,

$$f\left(1\right) = -1, \tag{1.2.9.6}$$

$$f'\left(1\right) = 1, \tag{1.2.9.7}$$

and hence

$$x_1 = 1 - \frac{-1}{1} = 2. \tag{1.2.9.8}$$

For the second iteration,

$$f(2) = 4 - 2 - 1 = 1, \tag{1.2.9.9}$$
$$f'(2) = 4 - 1 = 3. \tag{1.2.9.10}$$

Therefore,

$$x_2 = 2 - \frac{1}{3} \tag{1.2.9.11}$$
$$= \frac{5}{3} \tag{1.2.9.12}$$
$$= 1.6667\ldots \tag{1.2.9.13}$$

Thus, rounded off to two decimal places, $x_2 = 1.67$.

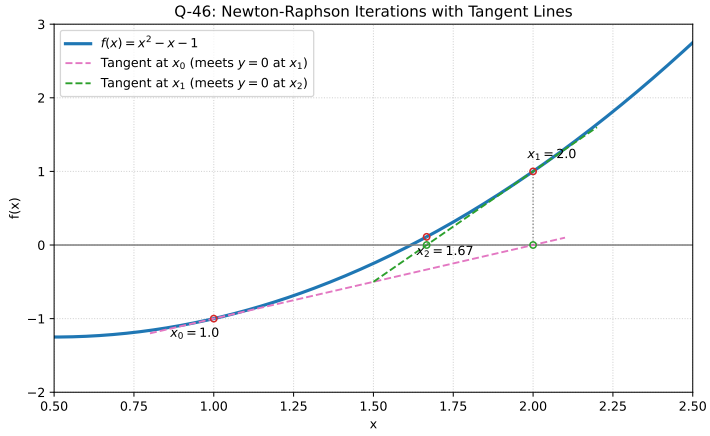


Fig. 1.2.9.1. Newton-Raphson iteration for $f(x) = x^2 - x - 1$

The code to verify this is:

codes/q46/main.py

1.2.10 The transfer function of a process is

$$G_p(s) = \frac{2e^{-s}}{(5s+1)^2}. \tag{1.2.10.1}$$

The process is fitted to a first-order plus dead-time $(FOPDT)$ model using the maximum slope method. If τ_m and θ_m are the fitted time constant and dead time, respectively, then τ_m/θ_m, rounded off to two decimal places, is _____. (GATE CH 2025)

The unit-step response of

$$\frac{1}{(\tau s+1)^2} \tag{1.2.10.2}$$

is

$$y(t) = 1 - \left(1 + \frac{t}{\tau}\right) e^{-t/\tau}, \tag{1.2.10.3}$$

$$\frac{dy}{dt} = \frac{t}{\tau^2} e^{-t/\tau}, \tag{1.2.10.4}$$

$$\frac{d^2y}{dt^2} = \frac{1}{\tau^2}\left(1 - \frac{t}{\tau}\right) e^{-t/\tau}. \tag{1.2.10.5}$$

Solution:
From (1.2.10.1), the process gain, time constant and dead time are

$$K = 2, \qquad \tau = 5 \text{ min}, \qquad \theta = 1 \text{ min}. \tag{1.2.10.6}$$

The factor e^{-s} represents a pure time delay of 1 min. Therefore, the actual process response is the delayed version of the second-order response in (1.2.10.3).
Thus, the actual step response is

$$y_p(t) = \begin{cases} 0, & t < 1, \\ 2\left[1 + \frac{t-1}{5}\right] e^{-(t-1)/5} \text{ subtracted from } 2, & t \geq 1. \end{cases} \tag{1.2.10.7}$$

Equivalently, writing the second branch directly,

$$y_p(t) = \begin{cases} 0, & t < 1, \\ 2\left[1 - \left(1 + \dfrac{t-1}{5}\right) e^{-(t-1)/5}\right], & t \geq 1. \end{cases} \tag{1.2.10.8}$$

For $t \geq 1$, differentiating (1.2.10.8) gives

$$\frac{dy_p}{dt} = \frac{2(t-1)}{25} e^{-(t-1)/5}, \tag{1.2.10.9}$$

$$\frac{d^2y_p}{dt^2} = \frac{2}{25}\left(1 - \frac{t-1}{5}\right) e^{-(t-1)/5}. \tag{1.2.10.10}$$

The maximum slope occurs at the inflection point. Hence, from (1.2.10.10),

$$\frac{d^2y_p}{dt^2} = 0. \tag{1.2.10.11}$$

Since the exponential term is non-zero,

$$1 - \frac{t-1}{5} = 0, \tag{1.2.10.12}$$

$$t - 1 = 5, \tag{1.2.10.13}$$

$$t_i = 6 \text{ min}. \tag{1.2.10.14}$$

Using the actual curve in (1.2.10.8), the output at the inflection point is

$$y_i = y_p(6) \tag{1.2.10.15}$$

$$= 2\left[1 - \left(1 + \frac{6-1}{5}\right) e^{-(6-1)/5}\right] \tag{1.2.10.16}$$

$$= 2\left(1 - \frac{2}{e}\right). \tag{1.2.10.17}$$

The maximum slope is obtained from (1.2.10.9):

$$R = \left.\frac{dy_p}{dt}\right|_{t=6} \tag{1.2.10.18}$$

$$= \frac{2(6-1)}{25} e^{-(6-1)/5} \tag{1.2.10.19}$$

$$= \frac{2}{5e}. \tag{1.2.10.20}$$

The maximum-slope method approximates the actual curve by the tangent at the inflection point. Therefore,

$$y - y_i = R(t - t_i). \tag{1.2.10.21}$$

Substituting (1.2.10.17), (1.2.10.20), and (1.2.10.14) into (1.2.10.21),

$$y - \left(2 - \frac{4}{e}\right) = \frac{2}{5e}(t - 6). \tag{1.2.10.22}$$

The fitted dead time θ_m is the time-axis intercept of this tangent. Therefore, putting $y = 0$ in (1.2.10.22),

$$-\left(2 - \frac{4}{e}\right) = \frac{2}{5e}(\theta_m - 6). \tag{1.2.10.23}$$

Hence,

$$\theta_m = 6 - \frac{2\left(1 - 2/e\right)}{2/\left(5e\right)} \tag{1.2.10.24}$$
$$= 6 - 5e + 10 \tag{1.2.10.25}$$
$$= 16 - 5e \tag{1.2.10.26}$$
$$\approx 2.4086 \text{ min.} \tag{1.2.10.27}$$

For the fitted FOPDT model,

$$G_m\left(s\right) = \frac{Ke^{-\theta_m s}}{\tau_m s + 1}, \tag{1.2.10.28}$$

the maximum slope is

$$R = \frac{K}{\tau_m}. \tag{1.2.10.29}$$

Therefore,

$$\tau_m = \frac{K}{R} \tag{1.2.10.30}$$
$$= \frac{2}{2/\left(5e\right)} \tag{1.2.10.31}$$
$$= 5e \tag{1.2.10.32}$$
$$\approx 13.5914 \text{ min.} \tag{1.2.10.33}$$

Finally, using (1.2.10.27) and (1.2.10.33),

$$\frac{\tau_m}{\theta_m} = \frac{5e}{16 - 5e} \tag{1.2.10.34}$$
$$= \frac{13.5914}{2.4086} \tag{1.2.10.35}$$
$$\approx 5.642. \tag{1.2.10.36}$$

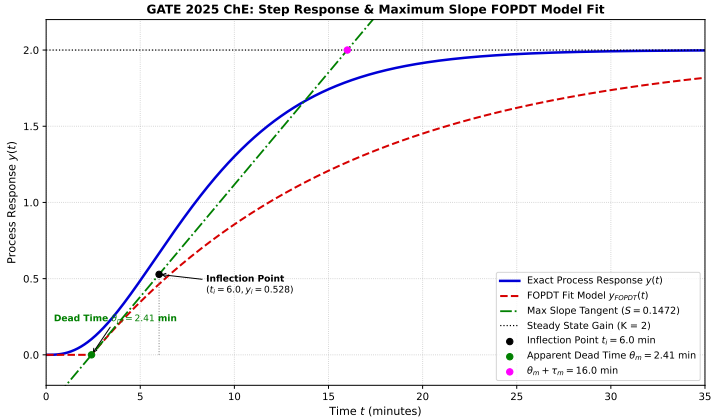


Fig. 1.2.10.1. Maximum-slope FOPDT approximation of the given process response

The code to verify this is:

codes/q60/main.py

1.2.11 The block diagram of a series cascade control system (with time in minutes) is shown in the figure. For $\tau_I = 8$ min and $K_c^s = 1$, the maximum value of K_c^m, below which the cascade control system is stable, is _____ (rounded off to the nearest integer). (GATE CH 2025)

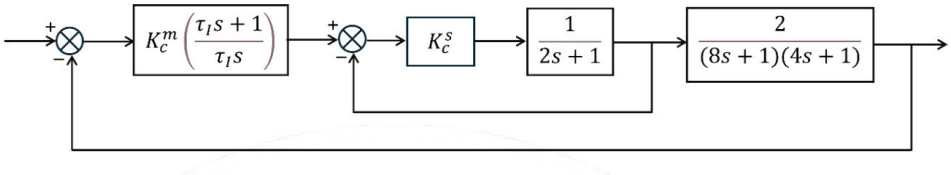


Fig. 1.2.11.1. Series cascade control system given in GATE CH 2025 Q62

Solution: The inner process is

$$G_1(s) = \frac{1}{2s+1}, \tag{1.2.11.1}$$

and the inner controller gain is $K_c^s = 1$. Therefore, the closed-loop inner transfer

function is

$$G_{\text{inner}}(s) = \frac{K_c^s G_1(s)}{1 + K_c^s G_1(s)} \tag{1.2.11.2}$$

$$= \frac{1/(2s+1)}{1 + 1/(2s+1)} \tag{1.2.11.3}$$

$$= \frac{1}{2(s+1)}. \tag{1.2.11.4}$$

The outer process is

$$G_2(s) = \frac{2}{(8s+1)(4s+1)}. \tag{1.2.11.5}$$

Hence the equivalent process for the outer loop is

$$G_{\text{eq}}(s) = G_{\text{inner}}(s) G_2(s) \tag{1.2.11.6}$$

$$= \frac{1}{(s+1)(8s+1)(4s+1)}. \tag{1.2.11.7}$$

The outer controller is

$$G_c(s) = K_c^m \frac{\tau_I s + 1}{\tau_I s}. \tag{1.2.11.8}$$

Since $\tau_I = 8$ min,

$$G_c(s) = K_c^m \frac{8s+1}{8s}. \tag{1.2.11.9}$$

Therefore, the open-loop transfer function of the outer loop is

$$G_c(s) G_{\text{eq}}(s) = K_c^m \frac{8s+1}{8s(s+1)(8s+1)(4s+1)} \tag{1.2.11.10}$$

$$= \frac{K_c^m}{8s(s+1)(4s+1)}. \tag{1.2.11.11}$$

For unity negative feedback, using (1.1.12.9), the characteristic equation is

$$1 + \frac{K_c^m}{8s(s+1)(4s+1)} = 0, \tag{1.2.11.12}$$

which gives

$$8s(s+1)(4s+1) + K_c^m = 0 \tag{1.2.11.13}$$

$$32s^3 + 40s^2 + 8s + K_c^m = 0. \tag{1.2.11.14}$$

Using the Routh-Hurwitz relations in (1.1.13.2), the Routh array is

$$\begin{pmatrix} 32 & 8 \\ 40 & K_c^m \\ \dfrac{40(8) - 32K_c^m}{40} & 0 \\ K_c^m & \end{pmatrix}. \tag{1.2.11.15}$$

For stability, every element in the first column must be positive. Thus,

$$8 - 0.8K_c^m > 0, \tag{1.2.11.16}$$

$$K_c^m > 0. \tag{1.2.11.17}$$

Therefore,

$$0 < K_c^m < 10. \tag{1.2.11.18}$$

Hence the maximum value of k_c^m below which the system is stable, rounded to the nearest integer, is 10.

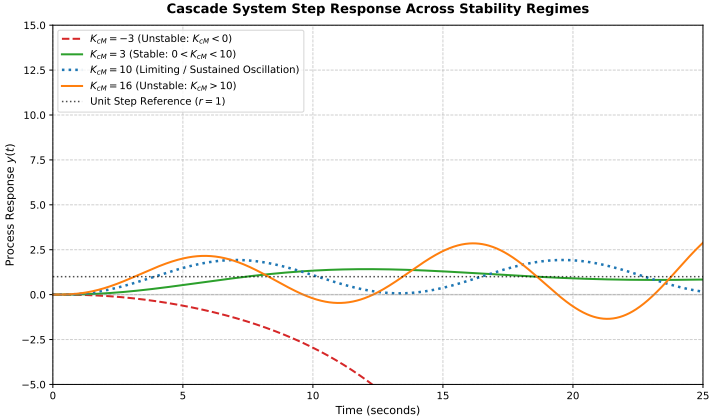


Fig. 1.2.11.2. Cascade-loop reduction and Routh-Hurwitz stability calculation for Q62

The code to verify this is:

codes/q62/main.py